

Q1. Debug the following programs**Code :**

```
print("Welcome to Cryptology Lab)
message = "Python Programming"
print(mesage)
num = input("Enter a number: ")
result = num + 10
print(result)
```

answer:**Error 1 : Syntax Error**

print("Welcome to Cryptology Lab) - The string starts with " but doesn't have a closing ".

Error 2 : Name error

```
message = "Python Programming"
print(mesage) - here messege "s" is missing .
```

error 3 : type error

```
num = input("Enter a number: ")
result = num + 10
"5" + 10 - here A string cannot be added to an integer.
```

Q2. Display the following exactly

Welcome to Cryptology Lab It's interesting to learn Python. Students, "Welcome to the Cryptology Lab".

```
Code : print("Welcome to Cryptology Lab")
print("It's interesting to learn Python.")
print('Students, "Welcome to the Cryptology Lab".')
```

[we're using **single quotes** around the third string so that the double quotes can appear normally.]

Q3. Arithmetic operations

We need:

- Addition
- Subtraction
- Multiplication
- Division
- Integer division
- Remainder
- Exponentiation

```
num1 = float(input("Enter first number: "))
```

```
num2 = float(input("Enter second number: "))
```

```
print("Addition:", num1 + num2)
```

```
print("Subtraction:", num1 - num2)
```

```
print("Multiplication:", num1 * num2)
```

```
print("Division:", num1 / num2)
```

```
print("Integer Division:", num1 // num2)
```

```
print("Remainder:", num1 % num2)
```

```
print("Exponentiation:", num1 ** num2)
```

output:

Addition: 13.0

Subtraction: 7.0

Multiplication: 30.0

Division: 3.3333333333333335

Integer Division: 3.0

Remainder: 1.0

Exponentiation: 1000.0

Q4. Floating-point number

Need to display:

1. Data type
2. Absolute value
3. Rounded to 2 decimal places

answer: **Code**

```
num = float(input("Enter a floating-point number: "))
```

```
print("Data type:", type(num))
```

```
print("Absolute value:", abs(num))
```

```
print("Rounded value:", round(num, 2))
```

Q5. Data Types and Collections

Question

Create variables containing examples of:

- int
- float
- complex
- str
- list
- tuple
- set
- dictionary

Then display the **value and data type** of each.

Answer code :

```
a = 10
```

```
b = 3.14
```

```
c = 2 + 3j
```

```
d = "Cryptology"
```

```
e = [10, 20, 30, 40, 50]
```

```
f = (10, 20, 30)
```

```
g = {10, 20, 30}
```

```
h = {"name": "Alice", "age": 20}
```

```
print("Integer:", a, "Type:", type(a))
```

```
print("Float:", b, "Type:", type(b))
```

```
print("Complex:", c, "Type:", type(c))
```

```
print("String:", d, "Type:", type(d))
```

```
print("List:", e, "Type:", type(e))
```

```
print("Tuple:", f, "Type:", type(f))
```

```
print("Set:", g, "Type:", type(g))
```

```
print("Dictionary:", h, "Type:", type(h))
```

Python supports different built-in data types such as integers, floating-point numbers, complex numbers, strings, lists, tuples, sets, and dictionaries. The `type()` function is used to identify the data type of a variable.

Q6:

Demonstrate experimentally that:

Strings are immutable, whereas lists are mutable.

Code :

```
# String - Immutable
```

```
message = "HELLO"
```

```
print("Original string:", message)
```

```
try:
```

```
    message[0] = "Y"
```

```
except TypeError:
```

```
    print("String cannot be modified because strings are immutable.")
```

List - Mutable

```
numbers = [10, 20, 30]
```

```
print("Original list:", numbers)
```

```
numbers[0] = 100
```

```
print("Modified list:", numbers)
```

output: Original string: HELLO

String cannot be modified because strings are immutable.

Original list: [10, 20, 30]

Modified list: [100, 20, 30]

Strings are immutable, which means their individual characters cannot be changed after creation. Lists are mutable, which means their elements can be modified after creation. The experiment shows that assigning a new value to a string element produces a `TypeError`, while assigning a new value to a list element works successfully.

Q7. List Operations

We need a list containing **five prime numbers**.

Then:

1. Display first 3
2. Display last element
3. Append another prime
4. Extend with 2 more primes
5. Display total number of elements

Code:

```
primes = [2, 3, 5, 7, 11]
```

```
print("Original list:", primes)
```

```
print("First three elements:", primes[:3])
```

```
print("Last element:", primes[-1])
```

```
primes.append(13)
```

```
print("After append:", primes)
```

```
primes.extend([17, 19])
```

```
print("After extend:", primes)
```

```
print("Total number of elements:", len(primes))
```

Q8 — String Slicing

Question

Display:

1. Length of the string
2. First five characters
3. Last six characters
4. Characters from positions 3 to 8
5. String in reverse order

Code:

```
message = "CRYPTOLOGY USING PYTHON"
```

```
print("Length of string:", len(message))
```

```
print("First five characters:", message[:5])
```

```
print("Last six characters:", message[-6:])
```

```
print("Characters from positions 3 to 8:", message[3:9])
```

```
print("Reverse order:", message[::-1])
```

Q9 — String Methods

Question

- Convert to lowercase
- Convert back to uppercase
- Check whether "PYTHON" exists
- Replace "PYTHON" with "PROGRAMMING"

Code:

```
Messege = "CRYPTOLOGY USING PYTHON"
Print("lowercase: " , messege.lower())
Print("upper: ",messege.upper())
Print("does python exist?: " , "PYTHON" in messege)
New_messege = messege.replace("PYTHON", "PROGRAMMING")
Print("After replacement: ", new_messege)
```

Output:

```
Lowercase: cryptology using python
Uppercase: CRYPTOLOGY USING PYTHON
Does PYTHON exist?: True
After replacement: CRYPTOLOGY USING PROGRAMMING
```

Q10 — Remove Spaces

Question

Remove all spaces from:

CRYPTOLOGY USING PYTHON

Code:

```
message = "CRYPTOLOGY USING PYTHON"

result = message.replace(" ", "")

print("String without spaces:", result)
```

output:

String without spaces: CRYPTOLOGYUSINGPYTHON

Q11 — Character and Unicode/ASCII Value , Using a loop, display every character and its corresponding value using ord().

Code:

```
message = "CRYPTOLOGY USING PYTHON"

for character in message:
```

```
print(character, "->", ord(character))
```

output:

C -> 67

R -> 82

Y -> 89

P -> 80

T -> 84

O -> 79

L -> 76

O -> 79

G -> 71

Y -> 89

-> 32

U -> 85

S -> 83

I -> 73

N -> 78

G -> 71

-> 32

P -> 80

Y -> 89

T -> 84

H -> 72

O -> 79

N -> 78

Q12 — chr() Conversion

Question

Convert:

65, 67, 73, 80, 72, 69, 82

into characters and combine them.

values = [65, 67, 73, 80, 72, 69, 82]


```
result = ""
```

```
for value in values:
```

```
    result = result + chr(value)
```

```
print("Characters:", result)
```

output:

Characters: CIPHER

Explanation: **chr() is the opposite of ord()**

```
ord("C")
```

→ 67

```
while:
```

```
chr(67)
```

→ "C"

Q13 — Plaintext Preprocessing

Question

Read a message from the user and:

1. Remove leading/trailing spaces
2. Convert to uppercase
3. Remove spaces between words

```
message = input("Enter a message: ")
```

```
message = message.strip()
```

```
message = message.upper()
```

```
message = message.replace(" ", "")
```

```
print("Processed message:", message)
```

Q14 — Character Grouping

Question

Read a string and divide it into blocks of **five characters**.

```
message = input("Enter a string: ")
```

```
for i in range(0, len(message), 5):
```

```
    print(message[i:i + 5])
```

Q15 — Padding ,Question

If the final block contains fewer than 5 characters, add X until the block contains 5 characters.

```
message = input("Enter a string: ")
```

```
for i in range(0, len(message), 5):
```

```
    block = message[i:i + 5]
```

```
    if len(block) < 5:
```

```
        block = block + "X" * (5 - len(block))
```

```
    print(block)
```

output: Enter a string: CRYPTOLOGYLAB

CRYPT

LOGY

LABXX

This technique is called **padding** and is commonly used to make data fit a required block size.

Q16 — Character Frequency

Given:

```
text = "CRYPTOLOGYISINTERESTING"
```

We need to calculate how many times each character appears and identify the most frequent character.

```
text = "CRYPTOLOGYISINTERESTING"
```

```
frequency = {}
```

```
for character in text:
```

```
if character in frequency:
    frequency[character] += 1
else:
    frequency[character] = 1

print("Character frequencies:")

for character, count in frequency.items():
    print(character, "->", count)

most_frequent = max(frequency, key=frequency.get)

print("Most frequently occurring character:", most_frequent)
print("Frequency:", frequency[most_frequent])
```

output:

Character frequencies:

C -> 1

R -> 2

Y -> 1

P -> 1

T -> 3

O -> 1

L -> 1

G -> 2

I -> 3

S -> 1

N -> 2

E -> 2

Most frequently occurring character: T

Frequency: 3

Frequency analysis is a basic technique used in **classical cryptanalysis**.

For example, if a ciphertext contains one letter much more frequently than others, that information can sometimes help attack a substitution cipher.

Q17 — Percentage Frequency

Now extend Q16.

The assignment gives:

Percentage Frequency = Frequency of Character / Total Number of Characters × 100

~ The formula in your assignment appears to have the numerator and denominator written in reverse. The mathematically correct formula is:

```
text = "CRYPTOLOGYISINTERESTING"
```

```
frequency = {}
```

```
for character in text:
```

```
    if character in frequency:
```

```
        frequency[character] += 1
```

```
    else:
```

```
        frequency[character] = 1
```

```
total = len(text)
```

```
print("Character Percentage Frequencies:")
```

```
for character, count in frequency.items():
```

```
    percentage = (count / total) * 100
```

```
    print(character, "->", round(percentage, 2), "%")
```

output:

Character Percentage Frequencies:

C -> 4.35 %

R -> 8.7 %

Y -> 4.35 %

P -> 4.35 %

T -> 13.04 %

O -> 4.35 %

L -> 4.35 %

G -> 8.7 %

I -> 13.04 %

S -> 4.35 %

N -> 8.7 %

E -> 8.7 %

Q18 — Modular Arithmetic

The question asks us to evaluate:

29 mod 26

55 mod 26

78 mod 26

-3 mod 26

-29 mod 26

```
print("29 mod 26 =", 29 % 26)
```

```
print("55 mod 26 =", 55 % 26)
```

```
print("78 mod 26 =", 78 % 26)
```

```
print("-3 mod 26 =", -3 % 26)
```

```
print("-29 mod 26 =", -29 % 26)
```

```
num = int(input("Enter an integer: "))
```

```
print("Number modulo 26:", num % 26)
```

Q19 — Alphabet Representation

We're using:

A = 0

B = 1

C = 2

...

Z = 25

So:

M = 12

We need both:

Character → Number

M → 12

Number → Character

Character to number

```
character = input("Enter an uppercase character: ")
```

```
number = ord(character) - ord('A')
```

```
print(character, "->", number)
```

Number to character

```
number = int(input("Enter number: "))
```

```
character = chr(number + ord('A'))
```

```
print(number, "->", character)
```

output: Enter an uppercase character: M

M -> 12

Enter number: 12

12 -> M

Q20 — Prime Numbers

We need a program that:

1. Determines whether a given number is prime.
2. Displays all primes within a user-specified range.

Code:

```
# Check whether a number is prime
```

```
num = int(input("Enter a number: "))
```

```
if num < 2:
```

```
    print(num, "is not a prime number")
```

```
else:
```

```
    is_prime = True
```

```
    for i in range(2, int(num ** 0.5) + 1):
```

```
        if num % i == 0:
```

```
            is_prime = False
```

```
            break
```

```
if is_prime:
```

```
    print(num, "is a prime number")
```

```
else:
```

```
    print(num, "is not a prime number")
```

```
# Display prime numbers in a range
```

```
start = int(input("Enter starting number: "))
```

```
end = int(input("Enter ending number: "))
```

```
print("Prime numbers in the range:")
```

```
for num in range(start, end + 1):
```

```
    if num < 2:
```

```
        continue
```

```
is_prime = True
```

```
for i in range(2, int(num ** 0.5) + 1):
```

```
    if num % i == 0:
```

```
        is_prime = False
```

```
        break
```

```
if is_prime:
```

```
    print(num, end=" ")
```

Q21 — GCD and Coprimes

We need to:

- Read two integers
- Find their GCD
- Determine whether they are coprime
- **What does coprime mean?**
 - Two numbers are **coprime** if their GCD is 1.
 - For example:
 - $\text{GCD}(15, 26) = 1$
 - Therefore, 15 and 26 are coprime.

```
import math
```

```
num1 = int(input("Enter first number: "))
```

```
num2 = int(input("Enter second number: "))
```

```
gcd = math.gcd(num1, num2)
```

```
print("GCD:", gcd)
```

```
if gcd == 1:
```

```
    print("The numbers are coprime.")
```

```
else:
```



```
print("The numbers are not coprime.")
```